Directed type theory in Arend

Formation, introduction, elimination, and computation rules

Repository snapshot `a7a01117506d` • 22 September 2026

This note extracts the directed core from the checked-out implementation of Arend. It follows the rule-and-equation presentation of §2.1 of *Theorem prover Arend* by Fedor Part, Valery Isaev, and Sergey Sinchuk. The source of the directed rules is this repository, especially `lib/Prelude.ard` and the typechecker; the linked paper describes the earlier HoTT-I core.

The extension keeps the ordinary interval and equality and adds a directed interval, morphisms, composition fillers, transport for families of groupoids, directed univalence, and a Rezk principle. The rules below describe the implemented primitives. Statements explicitly called *derived* are library constructions, not additional judgmental equalities.

1 Contexts, universes, and variance

Judgments. Write $\Gamma \vdash A\ \mathsf{type}$ for a well-formed type, $\Gamma \vdash a : A$ for typing, and $a \equiv b$ for judgmental equality. In contrast, $a =_A b$ is the ordinary path type. As in the paper, ambient contexts are omitted from equations. Substitution $A[i := r]$ is abbreviated to $A(r)$ when the bound variable is clear.

There are two kinds of variable: invariant $x : A$ and covariant $x :^{+} A$. The latter may occur only in positions checked covariantly. This is a syntactic restriction, even when A happens to be a groupoid. There is no contravariant binding annotation in this implementation. Write Γ° for the available invariant context: entering an invariant position hides the previously available covariant bindings. New covariant binders introduced inside that position are still permitted.

$$\frac{\Gamma^{\circ} \vdash A\ \mathsf{type}}{\Gamma, x : A\ \mathsf{ctx}}\ \text{(invariant extension)} \qquad \frac{\Gamma \vdash A\ \mathsf{type}}{\Gamma, x :^{+} A\ \mathsf{ctx}}\ \text{(covariant extension)}$$

All contexts in these schemata are assumed well formed. In particular, declaring an invariant parameter does not allow its type to depend on an earlier covariant parameter.

Universes. Type_u is the universe of groupoidal types of size u, and Cat_u permits categorical types. The relevant cumulative rules are

$$\frac{\Gamma \vdash A : \mathsf{Type}_u}{\Gamma \vdash A : \mathsf{Cat}_u}\ \text{(inclusion)} \qquad \Gamma \vdash \mathsf{Type}_u : \mathsf{Cat}_{u+1}, \qquad \Gamma \vdash \mathsf{Cat}_u : \mathsf{Cat}_{u+1}.$$

Both hierarchies are cumulative in u. The converse inclusion is absent; in particular $\mathsf{Type}_u : \mathsf{Type}_{u+1}$ is rejected. The finite homotopy levels remain: $\mathsf{Prop} \subseteq \mathsf{Set}_u \subseteq \mathsf{Type}_u$ and $\mathsf{Set}_u : (1\text{-}\mathsf{Type})_{u+1}$. Unsubscripted Type and Cat below follow the implementation's unindexed universe notation; they do not assert a particular finite size level.

Implementation: `BindingVariance`; `CheckTypeVisitor.clearCategoricalContext` and `visitParameter`; `ConstLevel.succ`; `Sort.compare`. Tests: `CatUniverseTest`, `CatPiTest`. Full paths are in Appendix A.

Dependent functions and covariant sums

Write $\Pi^{\nu}_{x:A}B$ for a dependent function type whose binder has mode $\nu \in \{0, +\}$; 0 is the ordinary invariant binder. Its introduction and application have the following shapes, assuming the displayed function type is well formed:

$$\frac{\Gamma, x :^{\nu} A \vdash b : B}{\Gamma \vdash \lambda^{\nu} x.b : \Pi^{\nu}_{x:A}B} \quad \text{(function introduction)}$$

$$\frac{\Gamma \vdash f : \Pi^{\nu}_{x:A}B \qquad \Gamma^{\nu} \vdash a : A}{\Gamma \vdash f\, a : B[x := a]} \quad \text{(application)} \qquad \Gamma^{+} = \Gamma, \quad \Gamma^{0} = \Gamma^{\circ}.$$

The usual function β and η equations apply, with variance respected. A covariant function $f : A \to^{+} B$ therefore acts on morphisms by

$$f_*(p) \;:\equiv\; \mathsf{dpath}(\lambda^{+} i.\, f(p \mathbin{@_d} i)).$$

An arbitrary invariant function $A \to B$ cannot be substituted for f here.

There is an additional formation restriction: when a Π-type is checked as a type expression, its domains are checked invariantly. Thus a covariantly varying A cannot in general be used to form $A \to^{+} B$ as an expression. Declaration telescopes have a special checking rule for function-typed parameters: a covariant parameter $f :^{+} A \to^{+} B$ may have a covariantly varying A, as in the primitive iso^{+} below. This permission propagates through successive covariant function domains, not through an invariant or nested negative domain. It does not supply a general covariance rule for function-type formation.

The covariant dependent sum is a categorical type former:

$$\frac{\Gamma \vdash A : \mathsf{Cat} \qquad \Gamma, x :^{+} A \vdash B : \mathsf{Cat}}{\Gamma \vdash \Sigma^{+}_{x:A}B : \mathsf{Cat}} \quad \text{(covariant sum)}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[a/x]}{\Gamma \vdash (a, b) : \Sigma^{+}_{x:A}B} \quad \text{(pair)} \qquad \frac{\Gamma \vdash s : \Sigma^{+}_{x:A}B}{\Gamma \vdash s.1 : A \qquad \Gamma \vdash s.2 : B[s.1/x]} \quad \text{(projections)}$$

$$(a, b).1 \equiv a, \qquad (a, b).2 \equiv b, \qquad (s.1, s.2) \equiv s.$$

Arend writes this as `\Sigma + (x : A) (B x)`: the mode is on the sum, not on individual fields. Ordinary `\Sigma` uses invariant fields and removes the categorical part of their universe levels; for example, for invariant $C : \mathsf{Cat}$, $\Sigma_{x:C}\mathsf{Nat} : \mathsf{Type}$ whereas $\Sigma^{+}_{x:C}\mathsf{Nat} : \mathsf{Cat}$. Covariant tuple elimination is supported for Σ^{+}, but not for ordinary Σ.

Implementation: `CheckTypeVisitor.checkPi`, `visitTypeParameter`, `visitSigmaParameter`, `visitTuple`; `DefinitionTypechecker.typecheckParameters`; `StdImplicitArgsInference`. Tests: `CatPiTest`, `CatSigmaTest`.

2 The two intervals and their paths

Ordinary paths. The HoTT-I interval $\mathbb{I}$ (`I`) with endpoints 0_I (`left`) and 1_I (`right`) is retained. For an invariant interval variable, $\mathsf{Path}(\lambda i.C, a, b)$ has introduction `path`, evaluation $p@i$, and the usual β, η, and endpoint equations. The implementation allows the family to take values in Cat. Ordinary transport is

$$\frac{\Gamma, i : \mathbb{I} \vdash C : \mathsf{Cat} \qquad \Gamma \vdash a : C(0_I) \qquad \Gamma^{\circ} \vdash r : \mathbb{I}}{\Gamma \vdash \mathsf{coe}(\lambda i.C, a, r) : C(r)} \quad \text{(ordinary transport)}$$

with $\mathsf{coe}(\lambda i.C, a, 0_I) \equiv a$ and $\mathsf{coe}(\lambda i.C, a, r) \equiv a$ when $i \notin \mathsf{FV}(C)$. This ordinary equality, rather than the directed morphism type, is used for the homotopies in Sections 4 and 5.

Directed interval. The new primitive is $\mathbb{D}$ (`DI`), with endpoints 0_d (`dleft`) and 1_d (`dright`):

$$\frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash \mathbb{D} : \mathsf{Set}_0}\ \text{(directed interval)} \qquad \frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash 0_d : \mathbb{D}}\ \text{(left endpoint)} \qquad \frac{\Gamma\ \mathsf{ctx}}{\Gamma \vdash 1_d : \mathbb{D}}\ \text{(right endpoint).}$$

A directed section uses a *covariant* interval variable. Given a covariant family C and endpoints a, b, its morphism type has rules

$$\frac{\Gamma, i :^+ \mathbb{D} \vdash C : \mathsf{Cat} \qquad \Gamma \vdash a : C(0_d) \qquad \Gamma \vdash b : C(1_d)}{\Gamma \vdash \mathsf{DPath}(\lambda^+ i.\, C, a, b) : \mathsf{Type}}\ \text{(directed path formation)}$$

$$\frac{\Gamma, i :^+ \mathbb{D} \vdash t : C}{\Gamma \vdash \mathsf{dpath}(\lambda^+ i.\, t) : \mathsf{DPath}(\lambda^+ i.\, C, t(0_d), t(1_d))}\ \text{(directed path introduction)}$$

$$\frac{\Gamma \vdash p : \mathsf{DPath}(\lambda^+ i.\, C, a, b) \qquad \Gamma \vdash r : \mathbb{D}}{\Gamma \vdash p \mathbin{@_d} r : C(r)}\ \text{(directed evaluation).}$$

The formation rule presupposes the well-formed family from its first premise; the introduction rule has that same presupposition. The defining equations are

$$\mathsf{dpath}(\lambda^+ i.\, t) \mathbin{@_d} r \equiv t[i := r], \qquad \mathsf{dpath}(\lambda^+ i.\, p \mathbin{@_d} i) \equiv p \quad (i \notin \mathsf{FV}(p)),$$
$$p \mathbin{@_d} 0_d \equiv a, \qquad p \mathbin{@_d} 1_d \equiv b.$$

For a constant family, put

$$\mathsf{Hom}_C(a, b) \ :\equiv\ \mathsf{DPath}(\lambda^+ i.\, C, a, b), \qquad \mathsf{idd}_a \ :\equiv\ \mathsf{dpath}(\lambda^+ i.\, a) : \mathsf{Hom}_C(a, a).$$

We also write $a \rightsquigarrow b$ when C is clear. Arend uses `a ~> b`, `idd`, and `p d@ i`; application `p i` elaborates to the same directed evaluation. Below, $p(i)$ abbreviates $p \mathbin{@_d} i$. Morphisms in a category form a groupoidal type: the conclusion is Type, not an arbitrary Cat.

Elimination of the interval. Matching on `DI` endpoints is allowed for an invariant parameter $i : \mathbb{D}$. With a well-formed invariant motive B, branches $b_0 : B(0_d)$ and $b_1 : B(1_d)$ give

$$\mathsf{case}_B(0_d; b_0, b_1) \equiv b_0, \qquad \mathsf{case}_B(1_d; b_0, b_1) \equiv b_1.$$

Such an eliminator takes its interval argument invariantly. Endpoint matching on $i :^+ \mathbb{D}$ is rejected, so this is not an eliminator for arbitrary directed sections. The partial boundary specifications of the primitive fillers and transport below are installed specially by the prelude implementation.

Implementation: `lib/Prelude.ard`: `DI`, `DPath`, `dpath`, `~>`, `idd`, `d@`; `AtExpression.make`; `CompareVisitor.comparePathEta`; `PatternTypechecking`; `ElimTypechecking`. Tests: `CatPreludeTest`.

3 Composition fillers

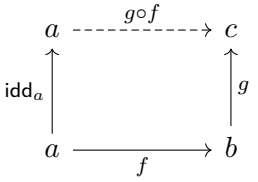
A two-dimensional filler

For composable morphisms, there is a chosen square with three prescribed edges:

$$\frac{\begin{array}{c}\Gamma \vdash C : \mathsf{Cat} \qquad \Gamma \vdash a, b, c : C \\ \Gamma \vdash f : a \rightsquigarrow b \qquad \Gamma \vdash g : b \rightsquigarrow c \\ \Gamma \vdash r, s : \mathbb{D}\end{array}}{\Gamma \vdash \mathsf{fill}_2(f, g, r, s) : C} \quad \texttt{(fill2)}$$

$$\begin{aligned}\mathsf{fill}_2(f, g, 0_d, j) &\equiv a, \\ \mathsf{fill}_2(f, g, 1_d, j) &\equiv g(j), \\ \mathsf{fill}_2(f, g, i, 0_d) &\equiv f(i).\end{aligned}$$

All parameters in this primitive, including the category, objects, arrows, and the two interval coordinates, are covariant. The square is oriented as follows; its fourth edge is the composite.



i : left to right,
j : bottom to top.

Derived composition:

$$g \circ f \;\coloneqq\; \mathsf{dpath}(\lambda^+ i.\, \mathsf{fill}_2(f, g, i, 1_d)) : a \rightsquigarrow c.$$

There is no further primitive reduction for the free edge $j = 1_d$. In particular, the unit and associativity laws are ordinary paths proved in the library, not extra judgmental equations for fill_2.

A three-dimensional filler

The primitive `fill3` fills a particular five-face boundary, with one collapsed edge. Its exact boundary is needed: it is not an unrestricted cubical horn-filling rule.

Fix $C : \mathsf{Cat}$ and seven objects $a, b, d, a', b', c', d' : C$ with arrows

$$\begin{array}{lll} ab : a \rightsquigarrow b, & bd : b \rightsquigarrow d, & ad : a \rightsquigarrow d, \\ a'b' : a' \rightsquigarrow b', & b'd' : b' \rightsquigarrow d', & a'c' : a' \rightsquigarrow c', \\ c'd' : c' \rightsquigarrow d', & aa' : a \rightsquigarrow a', & bb' : b \rightsquigarrow b', \\ ac' : a \rightsquigarrow c', & dd' : d \rightsquigarrow d'. & \end{array}$$

These are names of arrows, not compositions. Supply the following faces:

$$\begin{aligned} L &: \mathsf{DPath}(\lambda^+ k.\, (a \rightsquigarrow a'c'(k)), aa', ac'), \\ R &: \mathsf{DPath}(\lambda^+ k.\, (bd(k) \rightsquigarrow b'd'(k)), bb', dd'), \\ B &: \mathsf{DPath}(\lambda^+ i.\, (ab(i) \rightsquigarrow ad(i)), \mathsf{idd}_a, bd), \\ F &: \mathsf{DPath}(\lambda^+ i.\, (a'b'(i) \rightsquigarrow c'd'(i)), a'c', b'd'), \\ T &: \mathsf{DPath}(\lambda^+ i.\, (ab(i) \rightsquigarrow a'b'(i)), aa', bb'). \end{aligned}$$

Let ∂ denote exactly these objects, arrows, and five faces. Then

$$\frac{\Gamma \vdash \partial \text{ as above} \qquad \Gamma \vdash r, s, t : \mathbb{D}}{\Gamma \vdash \mathsf{fill}_3(\partial; r, s, t) : C} \quad (\texttt{fill3})$$

has the judgmental boundary equations

$$\begin{aligned} \mathsf{fill}_3(\partial; 0_d, j, k) &\equiv L(k)(j), & \mathsf{fill}_3(\partial; 1_d, j, k) &\equiv R(k)(j), \\ \mathsf{fill}_3(\partial; i, 0_d, k) &\equiv B(i)(k), & \mathsf{fill}_3(\partial; i, 1_d, k) &\equiv F(i)(k), \\ \mathsf{fill}_3(\partial; i, j, 0_d) &\equiv T(i)(j). \end{aligned}$$

The face types enforce agreement on all overlaps. In particular, $\mathsf{fill}_3(\partial; 0_d, 0_d, k) \equiv a$: this is the collapsed edge. The missing $k = 1_d$ face yields the derived term

$$\mathsf{dpath}(\lambda^+ i.\, \mathsf{dpath}(\lambda^+ j.\, \mathsf{fill}_3(\partial; i, j, 1_d))) : \mathsf{DPath}(\lambda^+ i.\, (ad(i) \rightsquigarrow c'd'(i)), ac', dd').$$

All the mathematical parameters above are covariant. The concrete prelude also has one invariant dummy argument of unit type, supplied as `fill3 ()`; it is omitted from the mathematical notation.

Derived coherence. The library uses fill_2 and fill_3 to prove contractibility of the type of a composite together with a composition triangle:

$$\Sigma^+_{h:a\rightsquigarrow c} \mathsf{DPath}(\lambda^+ i.\, (f(i) \rightsquigarrow h(i)), \mathsf{idd}_a, g).$$

Its chosen point is $(g \circ f, \mathsf{dpath}(\lambda^+ i.\, \mathsf{dpath}(\lambda^+ j.\, \mathsf{fill}_2(f, g, i, j))))$. Consequently the library proves $f \circ \mathsf{idd}_a = f$, $\mathsf{idd}_b \circ f = f$, and $h \circ (g \circ f) = (h \circ g) \circ f$ as ordinary equalities.

Implementation: `lib/Prelude.ard`: `fill2`, `fill3`; corresponding cases of `Prelude.update`. Derived constructions: `arend-lib/src/InfCat/Basics.ard`, especially `comp`, `segal2Prop`, `comp-char`, `comp_idd-left`, `comp_idd-right`, and `comp-assoc`.

4 Directed transport and directed univalence

Forward transport in a family of groupoids

$$\frac{\Gamma, i :^+ \mathbb{D} \vdash A : \mathsf{Type}_u \qquad \Gamma \vdash a : A(0_d) \qquad \Gamma \vdash r : \mathbb{D}}{\Gamma \vdash \mathsf{coe}^+(\lambda^+ i.\, A, a, r) : A(r)} \quad (\texttt{coe+})$$

$$\mathsf{coe}^+(\lambda^+ i.\, A, a, 0_d) \equiv a.$$

The family, starting element, and interval argument are covariant. The family must take values in Type_u; there is no primitive of this signature for arbitrary $A : \mathbb{D} \to^+ \mathsf{Cat}_u$, nor a primitive backward transport along a general directed arrow.

No general regularity equation. Unlike ordinary coe, the normalizer does *not* impose $\mathsf{coe}^+(\lambda^+ i.\, A, a, r) \equiv a$ for a constant family. Only the endpoint rule above and the iso^+ rule below are specified. This is an intentional distinction in this description of the implementation, irrespective of which propositional equalities can be derived.

A function determines a directed path of types

$$\frac{\Gamma \vdash A, B : \mathsf{Type}_u \qquad \Gamma \vdash f : A \to^+ B \qquad \Gamma \vdash r : \mathbb{D}}{\Gamma \vdash \mathsf{iso}^+(A, B, f, r) : \mathsf{Type}_u} \quad \texttt{(iso+)}$$

This uses the covariant declaration-parameter convention described in Section 1: A, B, f, r are all covariant parameters. Implicit A, B are normally suppressed. Its equations are

$$\mathsf{iso}^+(A, B, f, 0_d) \equiv A, \qquad \mathsf{iso}^+(A, B, f, 1_d) \equiv B,$$
$$\mathsf{coe}^+(\lambda^+ i.\, \mathsf{iso}^+(A, B, f, i), a, 1_d) \equiv f(a) \qquad (i \notin \mathsf{FV}(A, B, f)).$$

Only a forward function is required. No inverse or inverse homotopies occur in this directed principle. Define

$$\mathsf{ua}^+(f) \coloneqq \mathsf{dpath}(\lambda^+ i.\, \mathsf{iso}^+(A, B, f, i)) : A \rightsquigarrow B, \qquad \mathsf{tr}(p) \coloneqq \lambda^+ a.\, \mathsf{coe}^+(\lambda^+ i.\, p(i), a, 1_d).$$

Then $\mathsf{tr}(\mathsf{ua}^+(f)) \equiv f$ by the computation rule and function η.

The other composite is homotopic to the identity

For $A, B : \mathsf{Type}_u$ and $p : A \rightsquigarrow B$, the primitive `iso+_coe+` supplies a mixed directed/ordinary square:

$$\frac{\begin{array}{c}\Gamma \vdash A, B : \mathsf{Type}_u \qquad \Gamma \vdash p : A \rightsquigarrow B \\ \Gamma \vdash r : \mathbb{D} \qquad \Gamma^\circ \vdash s : \mathbb{I}\end{array}}{\Gamma \vdash \mathsf{iso}^+_{\mathsf{coe}^+}(p, r, s) : \mathsf{Type}_u} \quad \texttt{(iso+_coe+)}$$

The parameters A, B, p, r are covariant, while s is invariant. Its four boundary equations are

$$\mathsf{iso}^+_{\mathsf{coe}^+}(p, 0_d, j) \equiv A, \qquad \mathsf{iso}^+_{\mathsf{coe}^+}(p, 1_d, j) \equiv B,$$
$$\mathsf{iso}^+_{\mathsf{coe}^+}(p, i, 0_I) \equiv \mathsf{iso}^+(A, B, \mathsf{tr}(p), i), \qquad \mathsf{iso}^+_{\mathsf{coe}^+}(p, i, 1_I) \equiv p(i).$$

Thus the other round trip is witnessed by an *ordinary path*:

$$\mathsf{path}(\lambda j.\mathsf{dpath}(\lambda^+ i.\, \mathsf{iso}^+_{\mathsf{coe}^+}(p, i, j))) : \mathsf{ua}^+(\mathsf{tr}(p)) = p.$$

In an invariant context, these give a function/path correspondence $(A \to^+ B) \simeq \mathsf{Hom}_{\mathsf{Type}_u}(A, B)$. This correspondence is a consequence of the three primitives, with one judgmental round trip and one supplied homotopy; it is not a judgmental η rule for iso^+.

Implementation: `lib/Prelude.ard`: `coe+`, `iso+`, `iso+_coe+`; `Prelude.update`; `NormalizeVisitor.normalizeCoe`, including its exclusion of `COERCE_PLUS` from constant-family reduction. Tests: `CatPreludeTest.coePlusTest`, `isoPlusTest`, `isoPlusLevelError`, `isoCoePlusTest`.

5 Rezk completeness and the groupoidal fragment

The Rezk principle

Let $C : \mathsf{Cat}$, $x, y : C$, $f : x \rightsquigarrow y$, and $g : y \rightsquigarrow x$. Instead of merely asking for equations between composites, the primitive takes two composition triangles:

$$P : \mathsf{DPath}(\lambda^+ i.\, (f(i) \rightsquigarrow x), \mathsf{idd}_x, g), \qquad Q : \mathsf{DPath}(\lambda^+ i.\, (g(i) \rightsquigarrow y), \mathsf{idd}_y, f).$$

Writing $S_x \coloneqq \Sigma^+_{z:C} \mathsf{Hom}_C(x, z)$, the rule is

$$\frac{\begin{array}{c}\Gamma \vdash C : \mathsf{Cat} \qquad \Gamma \vdash x, y : C \\ \Gamma \vdash f : x \rightsquigarrow y \qquad \Gamma \vdash g : y \rightsquigarrow x \\ \Gamma \vdash P, Q \text{ of the displayed triangle types}\end{array}}{\Gamma \vdash \mathsf{rezk}(f, g, P, Q) : (x, \mathsf{idd}_x) =_{S_x} (y, f)} \quad (\texttt{rezk}).$$

All arguments are covariant. Projecting this ordinary path to its first component yields $x =_C y$; the second component identifies the chosen arrow with the arrow represented by that equality. Thus the primitive records more than a bare identification of objects.

Computation. No defining body or special judgmental computation rule is installed for rezk. Its ordinary path evaluations still satisfy the general endpoint equations:

$$\mathsf{rezk}(f, g, P, Q)@0_I \equiv (x, \mathsf{idd}_x), \qquad \mathsf{rezk}(f, g, P, Q)@1_I \equiv (y, f).$$

Using the composition-triangle characterization of Section 3, the library derives that an invertible morphism is represented by an ordinary equality. In particular, it proves the equivalence between $x =_C y$ and the type of isomorphisms from x to y.

Identity elimination for morphisms in a groupoid

When $A : \mathsf{Type}$, a morphism in a *constant* family admits identity elimination. A representative based form is

$$\frac{\begin{array}{c}\Gamma \vdash A : \mathsf{Type} \qquad \Gamma \vdash a : A \\ \Gamma, y : A, p : a \rightsquigarrow y \vdash B(y, p)\ \mathsf{type} \\ \Gamma \vdash b : B(a, \mathsf{idd}_a) \qquad \Gamma \vdash y : A \qquad \Gamma \vdash p : a \rightsquigarrow y\end{array}}{\Gamma \vdash J_d(B, b, y, p) : B(y, p)} \quad \text{(groupoidal directed identity elimination)}$$

$$J_d(B, b, a, \mathsf{idd}_a) \equiv b.$$

This schema is stated in the invariant fragment. In the syntax it is a definition by the pattern `idd`. The checker requires a constant ambient family whose type has a noncategorical sort and an endpoint variable that can be eliminated, with the usual dependency/occurs checks. It rejects the analogous general rule for $A : \mathsf{Cat}$, as well as an unrestricted fixed-endpoint K rule.

Ordinary identity elimination gives a map $(a =_C b) \to (a \rightsquigarrow b)$ for any $C : \mathsf{Cat}$. For $A : \mathsf{Type}$, the rule above also gives the reverse map, and the library proves

$$\mathsf{Hom}_A(a, b) \simeq (a =_A b).$$

This explains the groupoidal fragment without identifying directed morphisms with ordinary equality in a general category.

Implementation: `lib/Prelude.ard`: `rezk`; `Prelude.update`; `PatternTypechecking` and `CoreExpressionChecker` for `idd` patterns. Tests: `IddTest`, especially `jTest`, `kTest`, `notInTypeError`, and `inTypeOk`. Derived results: `InfCat/Basics.ard` and `InfCat/Rezk.ard`.

A Implementation qualifications and source map

Size of path types. The formation rule in Section 2 deliberately uses unindexed Type. In an invariant context the size of $\mathsf{DPath}(C, a, b)$ is computed from its family, with groupoidal homotopy level. Unlike ordinary Path, DPath does not lower a finite homotopy level. If the path arguments depend on currently available covariant variables, the elaborator forces both path forms to unindexed Type (infinite size and groupoidal homotopy level). It does not preserve the family's finite size in that case. For example, invariant $x : \mathsf{Nat}$ permits $x \rightsquigarrow x : \mathsf{Type}_0$, while covariant $x :^{+} \mathsf{Nat}$ permits $x \rightsquigarrow x : \mathsf{Type}$ but not that finite-size conclusion. This is implemented by `DefCallResult.isForcedInfinite` and `GetTypeVisitor.visitPathType`, and tested by `CatPreludeTest.dPathCovariantTest` and its variants.

Scope. This is a presentation of the directed core and the structural rules it needs. Arend's general inductive definitions, records, truncation, level proofs, termination, and elaboration have further restrictions; their full rule systems are outside this note. For example, covariant datatype parameters do not automatically permit constructors with conditions or truncated declarations. The ordinary univalence primitive `iso` also remains in the prelude, for groupoidal types. No normalization, canonicity, or model-theoretic completeness theorem is claimed here.

The following paths are relative to the repository root at commit `a7a01117506d`. The full identifier is

`a7a01117506d7c81c7d38b78feec5204af7ec949`.

The working tree had no tracked source modifications when inspected.

Java paths in the list below, unless prefixed by `api/`, are relative to `base/src/main/java/org/arend/`.

Primitive signatures and boundaries
- `lib/Prelude.ard`;
- `prelude/Prelude.java`, method `update` (the directed cases are adjacent).

Variance and type formation
- `api/src/main/java/org/arend/ext/core/context/BindingVariance.java`;
- `typechecking/visitor/CheckTypeVisitor.java`;
- `typechecking/visitor/DefinitionTypechecker.java`;
- `typechecking/implicitargs/StdImplicitArgsInference.java`.

Universe levels
- `api/src/main/java/org/arend/ext/core/level/ConstLevel.java`;
- `core/sort/Sort.java`;
- `core/sort/SortExpression.java`.

Path typing, evaluation, and conversion
- `typechecking/result/DefCallResult.java`;
- `core/expr/AtExpression.java`;
- `core/expr/visitor/GetTypeVisitor.java`;
- `core/expr/visitor/CompareVisitor.java`;
- `core/expr/visitor/NormalizeVisitor.java`.

Elimination restrictions
- `typechecking/patternmatching/PatternTypechecking.java`;
- `typechecking/patternmatching/ElimTypechecking.java`;
- `typechecking/doubleChecker/CoreExpressionChecker.java`.

Regression tests
- `src/test/java/org/arend/cat/` (`CatPreludeTest`, `CatUniverseTest`, `CatPiTest`, `CatSigmaTest`, and related tests);
- `src/test/java/org/arend/typechecking/patternmatching/IddTest.java`.

Derived categorical laws
- `arend-lib/src/InfCat/Basics.ard`;
- `arend-lib/src/InfCat/Rezk.ard`;
- `arend-lib/src/InfCat/Groupoid.ard`.

B Concrete syntax examples

These examples use only the prelude. ASCII `:+`, `->+`, `=>+`, and `\Sigma +` are the counterparts of the superscript-plus forms in `lib/Prelude.ard`. Expected types let the checker infer the variance of the unannotated lambda binders in these examples.

```
\func identity {C :+ \Cat} {a :+ C} : a ~> a
  => dpath (\lam _ => a)

\func map {C D :+ \Cat} (f :+ C ->+ D)
          {a b :+ C} (p :+ a ~> b) : f a ~> f b
  => dpath (\lam i => f (p i))

\func compose {C :+ \Cat} {a b c :+ C}
              (f :+ a ~> b) (g :+ b ~> c) : a ~> c
  => dpath (\lam i => fill2 f g i dright)

\func uaPlus.{u} {A B :+ \Type u} (f :+ A ->+ B) : A ~> B
  => dpath (\lam i => iso+ f i)

\func uaBeta.{u} {A B :+ \Type u}
                 (f :+ A ->+ B) (a :+ A)
  : coe+ (\lam i => iso+ f i) a dright = f a
  => idp

\func uaEta.{u} {A B :+ \Type u} (p :+ A ~> B)
  : dpath (\lam i =>
      iso+ (\lam a => coe+ (\lam k => p k) a dright) i) = p
  => path (\lam j => dpath (\lam i => iso+_coe+ p i j))

\func directedJ {A : \Type} {a : A}
  (B : \Pi {b : A} -> a ~> b -> \Type)
  (base : B idd) {b : A} (p : a ~> b) : B p \elim p
  | idd => base
```

The equations witnessed by `idp` in these examples are judgmental; `uaEta` instead explicitly constructs an ordinary path. The supplied PDF is compiled from this self-contained source. Rebuild with two runs of `pdflatex directed-theory.tex`; see the accompanying `README.md` for validation commands.

C A consistency counterexample

The inspected snapshot is inconsistent. The following prelude-only module defines a closed term of a datatype with no constructors. It was accepted by the repository's typechecker and its core double checker at commit `a7a01117506d`. It is also saved as `docs/directed/inconsistency.ard`.

```
\data Empty

\func Family (i : DI) : \Type0 \elim i
  | dleft => \Sigma
  | dright => Empty

\func collapse {i : DI} (p : dleft ~> i) : Family i \elim p
  | idd => ()

\func contradiction : Empty
  => collapse (dpath (\lam i => i))
```

The derivation uses three features together:

1. The prelude declares `DI` as a datatype with two nullary constructors. Its inferred sort is `\Set 0`, and `Prelude.update` does not replace that sort with a categorical one. Thus $\mathbb{D} : \mathsf{Set}_0 \subseteq \mathsf{Type}$.
2. Directed path introduction gives the canonical segment
$$s \coloneqq \mathsf{dpath}(\lambda^{+} i.\, i) : 0_d \rightsquigarrow 1_d.$$
Since $\mathbb{D}$ is classified as groupoidal, the `idd` pattern accepts the based eliminator `collapse`.
3. Invariant endpoint matching gives $\mathsf{Family}(0_d) \equiv \mathbf{1}$ and $\mathsf{Family}(1_d) \equiv \mathbf{0}$. The identity case of `collapse` is therefore the unit value $()$. Applying it to s gives a term of $\mathsf{Family}(1_d) \equiv \mathbf{0}$.

There are no user axioms, holes, recursive definitions, or additional library imports in this module. Neither composition fillers nor directed univalence nor the Rezk primitive is needed for the contradiction.

The decisive implementation locations are the `"DI"` case of `Prelude.update` and the `typeSort.getHLevel().isCat()` check for directed identity patterns in `PatternTypechecking`. A repair must break this combination of rules; rejecting groupoidal elimination on the directed interval would block this particular derivation. That observation alone is not a consistency proof for a repaired system.

The 234 passing regression tests and the examples in Appendix B remain valid reports of checker behavior. They do not establish logical consistency. This counterexample concerns the implemented snapshot; a revised directed theory requires a separate soundness argument.